Xi'an Jiaotong-Liverpool University
西交利物浦大学

Module Code	Examiner	Department	Tel
INT104		INT	

2nd SEMESTER 24-25 RESIT EXAMINATION

Undergraduate

Artificial Intelligence

TIME ALLOWED: *3.5 hours*

INSTRUCTIONS TO CANDIDATES

1. This is an open-book exam and the duration is 3.5 hours.
2. Total marks available are 100. This accounts for 100% of the final mark.
3. Answer all questions. Relevant and clear steps should be included in the answers.
4. Please use MCQ card delivered to answer MCQ questions. Please use answer booklet for answer other questions.
5. Only English solutions are accepted.
6. The use of calculator is allowed.
7. All paper-based materials are allowed. NO DICTIONARIES.

Section 2 Computation Questions (28 Marks)

19. Given the following 2D data points:

$$X = \{(2,3), (3,3), (2,4), (7,8), (8,8), (7,9)\}$$

Perform k-mean with $k = 2$ and the initial centroids of $C_1 = (2,3)$ and $C_2 = (8,8)$. Stop the demonstration of computation after two whole iterations if the process does not converge by the end of the second iteration.

Please use city block distance to measure the distance i.e., $D(a,b) = |x_a - x_b| + |y_a - y_b|$

(10 Marks)

20. The following samples are provided as a training dataset.

- Class α: $\{A(1,2), B(2,1), C(2,3)\}$
- Class β: $\{D(3,2), E(3,4), F(4,6), G(6,4), H(5,6)\}$

By applying kNN algorithm, answer the following questions.

- Given $k = 3$, which class does $Z(3,3)$ belong to?
- Given $k = 5$, which class does $Z(3,3)$ belong to?
- Explain why the classification changes when increasing k from 3 to 5.

When you calculate the distance between samples, please use city block distance i.e., $D(M,N) = |x_M - x_N| + |y_M - y_N|$.

(10 Marks)

21. A hospital evaluates the diagnostic test for a disease on 200 patients:

- 45 patients **actually infected** (Positive class)
- 155 patients **not infected** (Negative class)

Test Results:

- 35 infected patients were **correctly identified**
- 15 healthy patients were **wrongly flagged** as infected

Now, please draw a confusion matrix for this diagnostic test and find the accuracy of the diagnostic test.

(8 Marks)

Section 3 Programming Questions (36 Marks)

22. Write a Python script that detects the best value of `C` in a binary SVM classification. Assume the input data is `X` whose shape is `(1000, 2)`. Please remember to perform cross validation. The candidate `C` value is `[0.1, 0.5, 0.7]`.

The appendix of the exam provides a set of API that may be used in this question.

(18 Marks)

23. Write a Python script that compares the clustering results of K-means with the following initialisation methods: 1) totally random; 2) pick random samples. The data should be randomly generated and contains 1000 samples for 2 clusters, which is named as a variable `X`. `k` hence should be set to 2. Use silhouette index to evaluate the performance of clustering.

The appendix of the exam provides a set of API that may be used in this question.

(18 Marks)

Section 4 Appendix: Useful API

API: `train_test_split` (scikit-learn)

Function Signature

```
sklearn.model_selection.train_test_split(
*arrays,                    % Input arrays (e.g., X, y)
test_size=None,             % Test set size (float or int)
train_size=None,            % Training set size (float or int)
random_state=None,          % Seed for reproducibility
shuffle=True,               # Whether to shuffle data
stratify=None               # Stratify split based on labels
)
```

Key Parameters

- `*arrays`: Input arrays to split (e.g., features `X`, labels `y`).
- `test_size`: Proportion (0.0–1.0) or absolute number of test samples. Default: `0.25`.
- `train_size`: Complement of `test_size` if not specified.
- `random_state`: Seed for shuffling (e.g., 42 for reproducibility).
- `shuffle`: Whether to shuffle data before splitting (default: `True`).
- `stratify`: Array-like object (e.g., labels `y`) for stratified splitting.

Returns

- List of split arrays: `[X_train, X_test, y_train, y_test, ...]`.
- Number of outputs matches the number of input arrays (always in train-test order).

API: SVC (scikit-learn)

Class Signature

```
class sklearn.svm.SVC(
C=1.0,
kernel='rbf',
degree=3,
gamma='scale',
random_state=None
)
```

Key Parameters

- **C** (float, default=1.0): Regularization parameter balancing margin width and classification error
- **kernel** (str, default='rbf'): Kernel type specification. Valid options: 'linear', 'poly', 'rbf', 'sigmoid'
- **degree** (int, default=3): Degree of polynomial kernel (only for poly kernel)
- **gamma** (str/float, default='scale'): Kernel coefficient for 'rbf', 'poly', and 'sigmoid' kernels
- **random_state** (int, default=None): Seed for random number generation

Key Methods

- **fit(X, y)**: Trains the SVM model using training data
- **predict(X)**: Returns class labels for input samples
- **decision_function(X)**: Calculates confidence scores for samples
- **score(X, y)**: Returns mean accuracy on given test data

Kernel Functions

- Linear Kernel: $K(\mathbf{x}_i, \mathbf{x}_j) = \mathbf{x}_i \cdot \mathbf{x}_j$
- Polynomial Kernel: $K(\mathbf{x}_i, \mathbf{x}_j) = (\gamma \mathbf{x}_i \cdot \mathbf{x}_j + r)^d$

- RBF Kernel: $K(\mathbf{x}_i, \mathbf{x}_j) = \exp(-\gamma \|\mathbf{x}_i - \mathbf{x}_j\|^2)$
- Sigmoid Kernel: $K(\mathbf{x}_i, \mathbf{x}_j) = \tanh(\gamma \mathbf{x}_i \cdot \mathbf{x}_j + r)$

API: `accuracy_score` (scikit-learn)

Function Signature

```
sklearn.metrics.accuracy_score(
y_true,              % Ground truth labels
y_pred,              # Predicted labels
normalize=True,      % Return accuracy (True) or count (False)
sample_weight=None   % Sample weights
)
```

Key Parameters

- `y_true`: Array-like of true labels (shape: [n_samples])
- `y_pred`: Array-like of predicted labels (shape: [n_samples])
- `normalize`:
  - `True` (default): Return fraction of correct predictions
  - `False`: Return number of correct predictions
- `sample_weight`: Array-like of weights for samples (optional)

Returns

- Float between 0 and 1 if `normalize=True` (accuracy percentage)
- Integer count of correct predictions if `normalize=False`

API: `KMeans` (scikit-learn)

Class Signature

```
sklearn.cluster.KMeans(
n_clusters=8,            % Number of clusters
init='k-means++',        % Initialization method
n_init=10,               % Number of initializations
max_iter=300,            % Maximum iterations per run
tol=1e-4,                % Convergence tolerance
random_state=None        % Seed for reproducibility
)
```

Key Parameters

- `n_clusters`: Number of clusters to form (default: 8)
- `init`: Initialization method:
  - `'k-means++'` (default): Smart initialization
  - `'random'`: Random initialization
  - Array: Custom initial cluster centers
- `n_init`: Number of random initializations (default: 10)
- `max_iter`: Maximum iterations per initialization (default: 300)
- `tol`: Relative tolerance for convergence (default: 1e-4)
- `random_state`: Seed for random number generation

Key Methods

- `fit(X)`: Compute clustering on data matrix X
- `predict(X)`: Predict cluster labels for X
- `fit_predict(X)`: Compute clustering and return labels
- `transform(X)`: Transform X to cluster-distance space
- `score(X)`: Negative inertia value (higher is better)

Attributes

- `cluster_centers_`: Coordinates of cluster centers (shape: [n_clusters, n_features])
- `labels_`: Cluster labels for training data
- `inertia_`: Sum of squared distances to nearest cluster center
- `n_iter_`: Number of iterations run for best initialization

API: `silhouette_score` (scikit-learn)

Function Signature

```
sklearn.metrics.silhouette_score(
X,                      % Feature array or distance matrix
labels,                 # Cluster labels
metric='euclidean',     % Distance metric
sample_size=None,       % Subsample for large datasets
random_state=None,      % Reproducibility seed
**kwargs                % Metric-specific parameters
)
```

Key Parameters

- `X`: Feature matrix (n_samples × n_features) or distance matrix if `metric='precomputed'`
- `labels`: Array of cluster labels (shape: [n_samples])
- `metric`: Distance metric (default: 'euclidean'), can be:
  - Predefined metrics: 'cosine', 'manhattan', etc.
  - 'precomputed' if X is a distance matrix
- `sample_size`: Subsample dataset for faster computation (default: None)
- `random_state`: Seed for random subsampling (required if sample_size ≠ None)

Returns

- Float: Mean Silhouette Coefficient for all samples
- Range: [-1, 1], where:
  - 1: Perfect clustering
  - 0: Overlapping clusters
  - -1: Incorrect clustering

API: `sklearn.datasets.make_blobs` (scikit-learn)

Function Signature

```
sklearn.datasets.make_blobs(
n_samples=100,
centers=3,
n_features=2,
*,
cluster_std=1.0,
center_box=(-10.0, 10.0),
shuffle=True,
random_state=None,
return_centers=False
)
```

Key Parameters

- **n_samples** (int, default=100): Total number of data points generated
- **centers** (int/array-like, default=3): Number of clusters or fixed cluster centers
- **n_features** (int, default=2): Number of features/dimensions for samples
- **cluster_std** (float/array-like, default=1.0): Standard deviation of clusters
- **center_box** (tuple, default=(-10.0, 10.0)): Bounding box for cluster centers
- **shuffle** (bool, default=True): Shuffle samples after generation
- **return_centers** (bool, default=False): Return cluster centers

Returns

- **X** (array of shape [n_samples, n_features]): Feature matrix
- **y** (array of shape [n_samples]): Cluster labels
- **centers** (array, optional): Cluster centers (if return_centers=True)

Xi'an Jiaotong-Liverpool University
西交利物浦大學

END OF EXAM PAPER
THIS PAPER MUST NOT BE REMOVED FROM THE EXAMINATION ROOM